
Contrôle RVB et Effets LED avec Télécommande IR

Pilotage PWM d'une LED RGB, effets lumineux et luminosité adaptative

Darix SAMANI SIEWE
Rapport de projet – Systeme a Microcontrolleurs

1^{er} juillet 2026

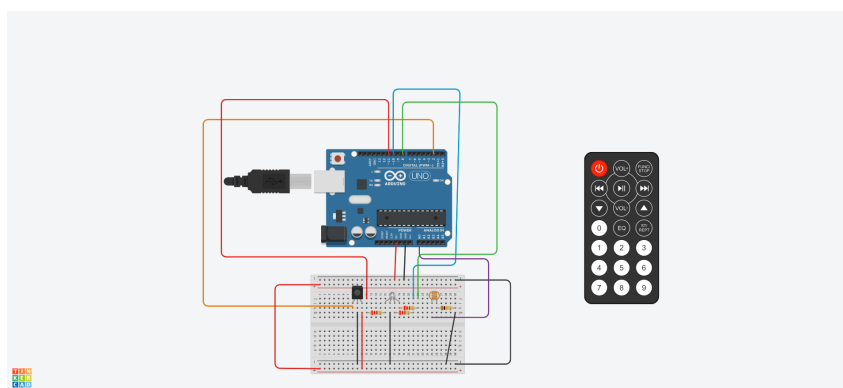


FIGURE 1 – Vue du montage du projet (capture Tinkercad ou photo réelle à insérer ici).

Table des matières

1	Introduction	2
2	Description du Projet	2
3	Composants Utilisés	2
3.1	Composants du cahier des charges (version complète)	2
3.2	Composants utilisés dans la simulation Tinkercad	2
4	Notions Électroniques Clés	3
5	Schéma du Montage	4
6	Code Arduino	4
7	Analyse et Discussion du Code	6
8	Ressources Complémentaires	7
9	Conclusion	7

1 Introduction

Ce rapport présente un projet de **contrôle avancé de LED RGB** à l'aide d'une carte **Arduino Uno**, piloté par une **télécommande infrarouge**. Le système propose plus de 15 effets lumineux (couleurs unies, dégradé arc-en-ciel, stroboscope, alternance) en agissant sur les trois canaux Rouge, Vert et Bleu par modulation de largeur d'impulsion (**PWM**). Un capteur de luminosité ambiante (**LDR**) permet en complément d'ajuster automatiquement l'intensité lumineuse afin d'éviter tout éblouissement dans l'obscurité.

Ce projet constitue une excellente introduction à la synthèse additive des couleurs, au décodage de protocoles infrarouges par interruption, ainsi qu'à l'utilisation de capteurs résistifs (LDR) au sein d'un diviseur de tension.

2 Description du Projet

Le système repose sur trois mécanismes complémentaires :

- le **décodage des commandes IR** reçues d'une télécommande, chaque touche étant associée à un mode ou une couleur précise (rouge, vert, bleu, arc-en-ciel, stroboscope, extinction) ;
- la **génération de couleurs** par combinaison des trois canaux PWM (R, V, B) de la LED RGB, selon le principe de la synthèse additive ;
- la **correction automatique de luminosité** grâce à la lecture d'un capteur LDR, qui module l'intensité globale de la LED en fonction de la lumière ambiante détectée.

Le programme implémente une **machine à états** à trois modes : le mode *Manuel* (couleur fixe choisie par une touche numérique), le mode *Arc-en-ciel* (transition fluide et continue entre les couleurs primaires), et le mode *Stroboscope* (clignotement blanc rapide). Chaque réception d'une commande IR valide peut faire basculer le système d'un mode à l'autre, illustrant une architecture événementielle réactive.

Objectif pédagogique : Ce projet permet de combiner trois compétences clés de l'électronique embarquée : la génération de signaux PWM pour piloter des intensités lumineuses, le décodage d'un protocole de télécommunication infrarouge, et l'exploitation d'un capteur analogique résistif pour une régulation automatique.

3 Composants Utilisés

3.1 Composants du cahier des charges (version complète)

3.2 Composants utilisés dans la simulation Tinkercad

Remarque : La liste exacte des composants Tinkercad (noms de référence U1, U2, etc.) n'a pas été transmise pour ce projet ; le tableau ci-dessus a donc été reconstitué à partir des broches effectivement utilisées dans le code (`pinIR=2`, `pinRouge=9`, `pinVert=10`, `pinBleu=11`, `pinLDR=A0`). N'hésite pas à me transmettre la liste précise des composants Tinkercad si tu souhaites que je corrige ce tableau.

4 Notions Électroniques Clés

lightgray Composant	Rôle dans le projet
LED RGB (x5)	Éléments lumineux pilotés en PWM pour produire les effets (rubans/groupes de LED)
Récepteur IR + Télécommande	Réception et décodage des commandes utilisateur
LDR (x2)	Mesure de la luminosité ambiante pour l'ajustement automatique
Résistances 220 Ω	Limitation du courant dans les canaux R, V, B des LED
Résistances 10 kΩ	Résistances de rappel (pull-down/pull-up) du diviseur de tension des LDR
Arduino Uno	Microcontrôleur assurant le décodage IR, la génération PWM et la logique d'effets

TABLE 1 – Composants prévus dans le cahier des charges initial du projet.

lightgray Réf- rence	Quantité	Composant
U1	1	Arduino Uno R3
U2	1	Récepteur infrarouge (IR)
—	1	Télécommande infrarouge (boutons numériques, Vol+/Vol-, Power)
D1 (LED RGB)	1	LED RGB (cathode/anode commune, pilotée sur les broches PWM 9, 10, 11)
LDR1	1	Photorésistance (capteur de luminosité ambiante, broche A0)
R1, R2, R3	3	Résistances 220 Ω (canaux R, V, B de la LED RGB)
R4	1	Résistance 10 k Ω (diviseur de tension de la LDR)

TABLE 2 – Composants utilisés dans la simulation Tinkercad d'après le code source fourni (broches 9, 10, 11 pour la LED RGB, broche 2 pour l'IR, broche A0 pour la LDR).

- **Synthèse additive des couleurs (modèle RVB) et PWM 8 bits (0–255)** : toute couleur visible peut être approchée par combinaison des trois couleurs primaires Rouge, Vert et Bleu. Sur Arduino, l'intensité de chaque canal est réglée par un signal **PWM** (Pulse Width Modulation) codé sur 8 bits (256 niveaux, de 0 à 255), où le rapport cyclique détermine la luminosité perçue de chaque LED.
- **Décodage de protocoles IR (NEC, Sony, RC5) par interruption** : les télécommandes infrarouges encodent les touches sous forme d'impulsions modulées selon un protocole standardisé (NEC étant le plus répandu). Le récepteur IR déclenche une **interruption matérielle** à chaque front du signal reçu, permettant à la bibliothèque (**IRremote**) de reconstituer la trame binaire correspondant à la touche pressée sans bloquer le programme principal.
- **Résistance de rappel (pull-down) et diviseur de tension pour LDR** : la LDR, dont la résistance varie avec la lumière reçue, est associée à une résistance fixe pour former un **diviseur de tension**. La tension lue sur la broche analogique varie ainsi en fonction de l'éclairement ambiant, permettant à l'Arduino de quantifier la luminosité via son convertisseur analogique-numérique.
- **Espace colorimétrique HSV pour la roue chromatique** : contrairement au modèle RVB (additif, peu intuitif pour générer des transitions fluides), l'espace **HSV** (Teinte,

Saturation, Valeur) permet de décrire une couleur par son angle sur la roue chromatique, facilitant le calcul d'une transition continue entre couleurs – une approche que l'effet arc-en-ciel du code approxime ici de façon simplifiée par interpolation linéaire directe entre canaux RVB.

5 Schéma du Montage

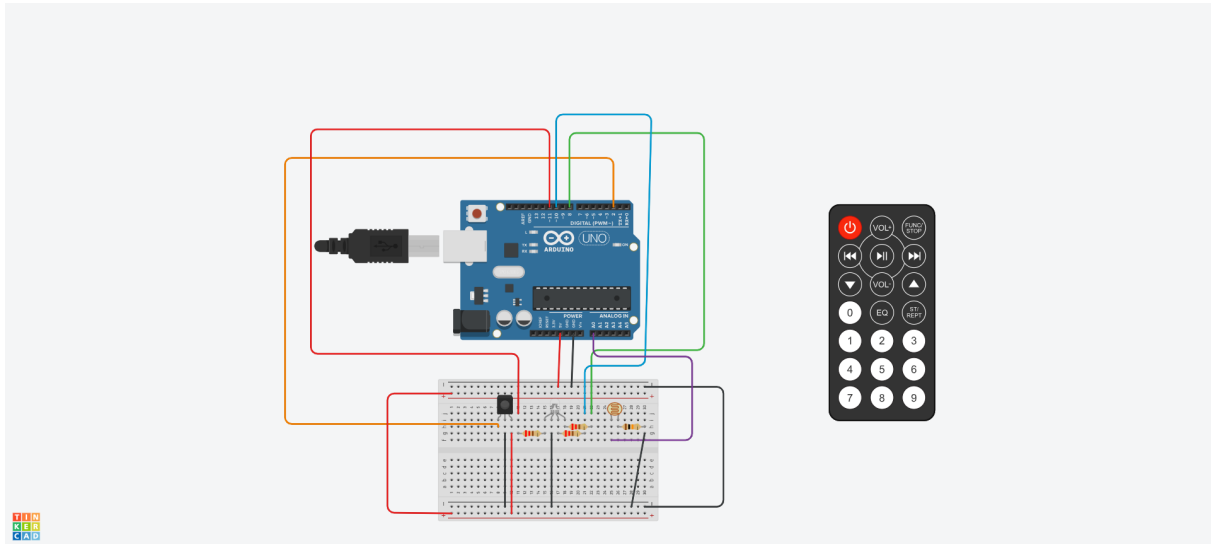


FIGURE 2 – Schéma de câblage du montage (Arduino Uno, récepteur IR, LED RGB sur broches PWM, LDR).

6 Code Arduino

Le programme assure le décodage des commandes de la télécommande IR, la lecture continue du capteur LDR pour l'ajustement de luminosité, ainsi que la génération des trois effets lumineux (manuel, arc-en-ciel, stroboscope).

```

1 #include <IRremote.h>
2
3 // --- Broches ---
4 const int pinIR = 2;
5 const int pinRouge = 9; // Broche PWM
6 const int pinVert = 10; // Broche PWM
7 const int pinBleu = 11; // Broche PWM
8 const int pinLDR = A0;
9
10 // --- Variables d'etat ---
11 int modeLumineux = 0; // 0=Manuel, 1=Arc-en-ciel, 2=Stroboscope
12 int rVal = 0, vVal = 0, bVal = 0;
13 float facteurLuminosite = 1.0;
14
15 void setup() {
16   Serial.begin(9600);
17
18   // Initialisation du récepteur IR
19   IrReceiver.begin(pinIR, ENABLE_LED_FEEDBACK);
20
21   // Configuration des broches de la LED RVB
22   pinMode(pinRouge, OUTPUT);

```

```

23   pinMode(pinVert, OUTPUT);
24   pinMode(pinBleu, OUTPUT);
25 }
26
27 void loop() {
28   // 1. Lecture automatique de la luminosité ambiante (LDR)
29   int valeurLDR = analogRead(pinLDR);
30   // Plus il fait noir, plus on baisse l'intensité de la LED pour ne pas
    blouir
31   facteurLuminosite = map(valeurLDR, 0, 1023, 10, 100) / 100.0;
32
33   // 2. Vérification des commandes de la télécommande IR
34   if (IrReceiver.decode()) {
35     unsigned long commande = IrReceiver.decodedIRData.decodedRawData;
36     Serial.print("Code IR reçu : ");
37     Serial.println(commande, HEX);
38
39     // Décodage des touches spécifiques de la télécommande Tinkercad
40     switch(IrReceiver.decodedIRData.command) {
41       case 0x10: // Touche "1" -> Mode Manuel Rouge
42         modeLumineux = 0; rVal = 255; vVal = 0; bVal = 0;
43         break;
44       case 0x11: // Touche "2" -> Mode Manuel Vert
45         modeLumineux = 0; rVal = 0; vVal = 255; bVal = 0;
46         break;
47       case 0x12: // Touche "3" -> Mode Manuel Bleu
48         modeLumineux = 0; rVal = 0; vVal = 0; bVal = 255;
49         break;
50       case 0x04: // Touche "Vol-" -> Mode Arc-en-ciel (Fondu)
51         modeLumineux = 1;
52         break;
53       case 0x06: // Touche "Vol+" -> Mode Stroboscope
54         modeLumineux = 2;
55         break;
56       case 0x00: // Touche "Power" -> éteindre tout
57         modeLumineux = 0; rVal = 0; vVal = 0; bVal = 0;
58         break;
59     }
60     IrReceiver.resume(); // Recommencer à écouter le capteur IR
61   }
62
63   // 3. Gestion des effets lumineux
64   if (modeLumineux == 0) {
65     // Mode Manuel classique
66     appliquerCouleur(rVal, vVal, bVal);
67   }
68   else if (modeLumineux == 1) {
69     // Mode Arc-en-ciel (Fondu de couleurs simplifié)
70     effetArcEnCiel();
71   }
72   else if (modeLumineux == 2) {
73     // Mode Stroboscope blanc flash
74     appliquerCouleur(255, 255, 255);
75     delay(50);
76     appliquerCouleur(0, 0, 0);
77     delay(50);
78   }
79 }
80
81 // Fonction pour envoyer les signaux PWM aux canaux de la LED RVB
82 void appliquerCouleur(int r, int v, int b) {
83   // On applique le facteur de correction de la LDR
84   analogWrite(pinRouge, r * facteurLuminosite);

```

```

85   analogWrite(pinVert, v * facteurLuminosite);
86   analogWrite(pinBleu, b * facteurLuminosite);
87 }
88
89 // Effet de transition fluide (Roue chromatique)
90 void effetArcEnCiel() {
91   for (int i = 0; i < 255; i++) {
92     if (modeLumineux != 1) return; // Quitter si le mode change
93     appliquerCouleur(255 - i, i, 0); // Transition Rouge vers Vert
94     delay(10);
95   }
96   for (int i = 0; i < 255; i++) {
97     if (modeLumineux != 1) return;
98     appliquerCouleur(0, 255 - i, i); // Transition Vert vers Bleu
99     delay(10);
100  }
101  for (int i = 0; i < 255; i++) {
102    if (modeLumineux != 1) return;
103    appliquerCouleur(i, 0, 255 - i); // Transition Bleu vers Rouge
104    delay(10);
105  }
106 }

```

Listing 1 – Code source Arduino du contrôle RVB avec télécommande IR (simulation Tinkercad).

7 Analyse et Discussion du Code

Le code peut être décomposé en cinq blocs fonctionnels principaux :

1. **Déclarations et initialisation (setup)** : les broches PWM 9, 10 et 11 sont assignées respectivement aux canaux Rouge, Vert et Bleu de la LED RGB, et configurées en sortie. Le récepteur IR est initialisé sur la broche 2 via `IrReceiver.begin()`, avec retour visuel activé (`ENABLE_LED_FEEDBACK`).
2. **Lecture de la luminosité ambiante (LDR)** : à chaque itération de la boucle, la valeur analogique du LDR (0–1023) est convertie en un **facteur de luminosité** compris entre 0,10 et 1,00 via `map()`. Ce facteur est ensuite appliqué multiplicativement à chaque canal PWM, ce qui réduit automatiquement l'intensité des LED dans l'obscurité – une approche simple mais efficace de régulation adaptative.
3. **Décodage des commandes IR** : la méthode `IrReceiver.decode()` renvoie `true` dès qu'une trame complète a été reçue et décodée. Le code de commande (`IrReceiver.decodedIRData.command`) est comparé via un `switch` à des valeurs hexadécimales correspondant aux touches de la télécommande virtuelle Tinkercad, chacune déclenchant un changement d'état (`modeLumineux`) et/ou de couleur cible (`rVal`, `vVal`, `bVal`). L'appel à `IrReceiver.resume()` est indispensable pour réarmer le récepteur et permettre la réception de la commande suivante.
4. **Machine à états des modes lumineux** : selon la valeur de `modeLumineux` (0, 1 ou 2), le programme applique soit une couleur fixe (mode Manuel), soit l'effet arc-en-ciel (`effetArcEnCiel()`), soit un clignotement stroboscopique blanc/noir avec un délai de 50 ms. Cette structure conditionnelle simple constitue une machine à états minimaliste, suffisante pour ce nombre réduit de modes.
5. **Fonctions utilitaires (appliquerCouleur, effetArcEnCiel)** : la fonction `appliquerCouleur()` centralise l'écriture PWM sur les trois canaux en appliquant le facteur de luminosité, évitant la duplication de code. La fonction `effetArcEnCiel()` réalise une transition cyclique Rouge→Vert→Bleu→Rouge par interpolation linéaire sur 255 pas, avec une vérification de sortie anticipée (`if (modeLumineux != 1) return;`) permettant de quitter l'effet dès

qu’une nouvelle commande change de mode, malgré l’usage de boucles bloquantes avec `delay(10)`.

Points forts du code :

- Centralisation de l’écriture PWM dans une fonction unique (`appliquerCouleur`), incluant systématiquement la correction de luminosité ambiante.
- Vérification de sortie anticipée dans `effetArcEnCiel()`, qui permet à l’utilisateur de changer de mode même pendant l’exécution d’une transition longue.
- Utilisation cohérente et lisible du `switch` pour associer chaque touche de télécommande à une action précise.

Pistes d’amélioration possibles :

- Remplacer les `delay(10)` et `delay(50)` par une gestion non bloquante via `millis()`, afin que la lecture de la LDR et la détection IR restent réactives en continu pendant les effets, sans interruption perceptible de la réception des commandes.
- Implémenter une véritable conversion **HSV vers RVB** pour l’effet arc-en-ciel, offrant une roue chromatique continue et homogène plutôt que trois interpolations linéaires successives entre paires de canaux.
- Ajouter une gestion d’erreur pour les codes IR non reconnus (actuellement ignorés silencieusement par le `switch` sans `default`), par exemple un retour visuel ou un message de débogage.
- Étendre la structure existante (machine à états + table de correspondance touche/action) pour intégrer facilement les effets supplémentaires visés par le cahier des charges (plus de 15 effets au total), en généralisant le `switch` vers un tableau de fonctions.

8 Ressources Complémentaires

- **Vidéo** – *Arduino RGB LED Control with IR Remote : Full Tutorial*
<https://www.youtube.com/watch?v=qS11NpB5b-M>
- **Vidéo** – *Effets LED avancés Arduino (francophone complet)*
<https://www.youtube.com/watch?v=S5Q2BuSJ10Q>

9 Conclusion

Ce projet illustre de manière concrète comment combiner génération de signaux PWM, décodage d’un protocole de télécommunication infrarouge et exploitation d’un capteur analogique au sein d’un même système réactif. L’architecture logicielle, bien que limitée à trois modes dans cette version, repose sur une structure modulaire (fonctions `appliquerCouleur` et machine à états par `switch`) directement extensible vers le large éventail d’effets lumineux (plus de 15) visé par le cahier des charges complet, notamment via l’introduction de l’espace colorimétrique HSV pour des transitions chromatiques plus naturelles.